

Studentu vertinimo sistema

Generated by Doxygen 1.13.2

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Studentas Class Reference	7
4.1.1 Constructor & Destructor Documentation	8
4.1.1.1 Studentas() [1/5]	8
4.1.1.2 Studentas() [2/5]	8
4.1.1.3 Studentas() [3/5]	8
4.1.1.4 Studentas() [4/5]	8
4.1.1.5 Studentas() [5/5]	8
4.1.1.6 ~Studentas()	9
4.1.2 Member Function Documentation	9
4.1.2.1 egzaminas()	9
4.1.2.2 galutinis_mediana()	9
4.1.2.3 galutinis_vidurkis()	9
4.1.2.4 ivedimas_generuojant_viska()	9
4.1.2.5 ivedimas_pazymiu_generavimu()	9
4.1.2.6 ivedimas_ranka()	9
4.1.2.7 operator=() [1/2]	9
4.1.2.8 operator=() [2/2]	9
4.1.2.9 paskaiciuoti_gal()	10
4.1.2.10 paskaiciuoti_vid_ir_med()	10
4.1.2.11 pavarde()	10
4.1.2.12 spausdinti()	10
4.1.2.13 sugeneruoti_lyti()	10
4.1.2.14 sugeneruoti_pavarde()	10
4.1.2.15 sugeneruoti_varda()	10
4.1.2.16 vardas()	10
4.1.3 Friends And Related Symbol Documentation	10
4.1.3.1 operator<<	10
4.1.3.2 operator>>	11
4.2 Vector< T, Alloc > Class Template Reference	11
4.2.1 Member Typedef Documentation	12
4.2.1.1 allocator_type	12
4.2.1.2 const_iterator	12
4.2.1.3 const_pointer	13

4.2.1.4 const_reference	13
4.2.1.5 const_reverse_iterator	13
4.2.1.6 difference_type	13
4.2.1.7 iterator	13
4.2.1.8 pointer	13
4.2.1.9 reference	13
4.2.1.10 reverse_iterator	13
4.2.1.11 size_type	13
4.2.1.12 value_type	14
4.2.2 Constructor & Destructor Documentation	14
4.2.2.1 Vector() [1/6]	14
4.2.2.2 Vector() [2/6]	14
4.2.2.3 Vector() [3/6]	14
4.2.2.4 Vector() [4/6]	14
4.2.2.5 Vector() [5/6]	14
4.2.2.6 Vector() [6/6]	14
4.2.2.7 ~Vector()	15
4.2.3 Member Function Documentation	15
4.2.3.1 assign()	15
4.2.3.2 at() [1/2]	15
4.2.3.3 at() [2/2]	15
4.2.3.4 back() [1/2]	15
4.2.3.5 back() [2/2]	15
4.2.3.6 begin() [1/2]	15
4.2.3.7 begin() [2/2]	15
4.2.3.8 capacity()	16
4.2.3.9 cbegin()	16
4.2.3.10 cend()	16
4.2.3.11 clear()	16
4.2.3.12 crbegin()	16
4.2.3.13 crend()	16
4.2.3.14 data() [1/2]	16
4.2.3.15 data() [2/2]	16
4.2.3.16 emplace()	17
4.2.3.17 emplace_back()	17
4.2.3.18 empty()	17
4.2.3.19 end() [1/2]	17
4.2.3.20 end() [2/2]	17
4.2.3.21 erase() [1/2]	17
4.2.3.22 erase() [2/2]	17
4.2.3.23 front() [1/2]	17
4.2.3.24 front() [2/2]	18

4.2.3.25 insert() [1/2]	18
4.2.3.26 insert() [2/2]	18
4.2.3.27 operator=() [1/2]	18
4.2.3.28 operator=() [2/2]	18
4.2.3.29 operator[]() [1/2]	18
4.2.3.30 operator[]() [2/2]	18
4.2.3.31 pop_back()	18
4.2.3.32 push_back() [1/2]	19
4.2.3.33 push_back() [2/2]	19
4.2.3.34 rbegin() [1/2]	19
4.2.3.35 rbegin() [2/2]	19
4.2.3.36 rend() [1/2]	19
4.2.3.37 rend() [2/2]	19
4.2.3.38 reserve()	19
4.2.3.39 resize()	19
4.2.3.40 shrink_to_fit()	20
4.2.3.41 size()	20
4.2.3.42 swap()	20
4.3 Zmogus Class Reference	20
4.3.1 Constructor & Destructor Documentation	21
4.3.1.1 Zmogus() [1/2]	21
4.3.1.2 Zmogus() [2/2]	21
4.3.1.3 ~Zmogus()	21
4.3.2 Member Function Documentation	21
4.3.2.1 pavarde()	21
4.3.2.2 spausdinti()	21
4.3.2.3 vardas()	21
4.3.3 Member Data Documentation	21
4.3.3.1 pav_	21
4.3.3.2 var_	21
5 File Documentation	23
5.1 vector/failu_funkcijos.cpp File Reference	23
5.1.1 Function Documentation	23
5.1.1.1 buferio_apdorojimas()	23
5.1.1.2 failo_generavimas()	23
5.1.1.3 failo_pav_gavimas()	23
5.1.1.4 ivedimas_failu()	24
5.1.1.5 rasti_nd_skaiciu_faile()	24
5.2 vector/failu_funkcijos.h File Reference	24
5.2.1 Function Documentation	24
5.2.1.1 buferio_apdorojimas()	24

5.2.1.2 failo_generavimas()	24
5.2.1.3 failo_pav_gavimas()	24
5.2.1.4 ivedimas_failu()	25
5.2.1.5 rasti_nd_skaiciu_faile()	25
5.3 failu_funkcijos.h	25
5.4 vector/main.cpp File Reference	25
5.4.1 Function Documentation	25
5.4.1.1 main()	25
5.5 vector/mano_lib.h File Reference	26
5.6 mano_lib.h	26
5.7 vector/spausdinimo_funkcijos.cpp File Reference	26
5.7.1 Function Documentation	27
5.7.1.1 eigos_parinktis()	27
5.7.1.2 spausdinimas_atskiras()	27
5.7.1.3 spausdinimas_kartu()	27
5.8 vector/spausdinimo_funkcijos.h File Reference	27
5.8.1 Function Documentation	27
5.8.1.1 eigos_parinktis()	27
5.8.1.2 spausdinimas_atskiras()	27
5.8.1.3 spausdinimas_kartu()	27
5.9 spausdinimo_funkcijos.h	28
5.10 vector/studentas.cpp File Reference	28
5.10.1 Function Documentation	28
5.10.1.1 lyginti_pagal_mediana()	28
5.10.1.2 lyginti_pagal_pavarde()	28
5.10.1.3 lyginti_pagal_varda()	28
5.10.1.4 lyginti_pagal_vidurki()	29
5.10.1.5 operator<<()	29
5.10.1.6 operator>>()	29
5.10.1.7 pasirink_rusiavimas()	29
5.10.1.8 rusiuoti_grupemis()	29
5.11 vector/studentas.h File Reference	29
5.11.1 Function Documentation	30
5.11.1.1 lyginti_pagal_mediana()	30
5.11.1.2 lyginti_pagal_pavarda()	30
5.11.1.3 lyginti_pagal_varda()	30
5.11.1.4 lyginti_pagal_vidurki()	30
5.11.1.5 pasirink_rusiavimas()	30
5.11.1.6 rusiuoti_grupemis()	30
5.12 studentas.h	31
5.13 vector/Vector.hpp File Reference	31
5.13.1 Function Documentation	32

5.13.1.1 operator"!=()	32
5.13.1.2 operator<<()	32
5.13.1.3 operator==()	32
5.13.1.4 swap()	32
5.14 Vector.hpp	33
5.15 vector/zmogus.h File Reference	36
5.16 zmogus.h	36
Index	37

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Vector< T, Alloc >	11
Zmogus	20
Studentas	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	7
Vector< T, Alloc >	11
Zmogus	20

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

vector/ failu_funkcijos.cpp	23
vector/ failu_funkcijos.h	24
vector/ main.cpp	25
vector/ mano_lib.h	26
vector/ spausdinimo_funkcijos.cpp	26
vector/ spausdinimo_funkcijos.h	27
vector/ studentas.cpp	28
vector/ studentas.h	29
vector/ Vector.hpp	31
vector/ zmogus.h	36

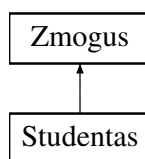
Chapter 4

Class Documentation

4.1 Studentas Class Reference

```
#include <studentas.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- [Studentas](#) ()
- [Studentas](#) (const string &[vardas](#), const string &[pavarde](#), const [Vector](#)< int > &paz, int [egzaminas](#))
- [Studentas](#) (std::istream &is, int sk)
- [Studentas](#) (const [Studentas](#) &other)
- [Studentas](#) ([Studentas](#) &&other) noexcept
- [Studentas](#) & [operator=](#) (const [Studentas](#) &other)
- [Studentas](#) & [operator=](#) ([Studentas](#) &&other) noexcept
- [~Studentas](#) ()
- void [ivedimas_ranka](#) ()
- void [ivedimas_pazymiu_generavimu](#) ()
- void [ivedimas_generuojant_viska](#) ()
- void [paskaiciuoti_vid_ir_med](#) ()
- void [paskaiciuoti_gal](#) ()
- void [sugeneruoti_lyti](#) ()
- void [sugeneruoti_varda](#) ()
- void [sugeneruoti_pavarde](#) ()
- string [vardas](#) () const
- string [pavarde](#) () const
- int [egzaminas](#) () const
- double [galutinis_vidurkis](#) () const
- double [galutinis_mediana](#) () const
- void [spausdinti](#) (std::ostream &os) const override

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()=default
- [Zmogus](#) (const string &[vardas](#), const string &[pavarde](#))
- virtual [~Zmogus](#) ()

Friends

- std::istream & [operator>>](#) (std::istream &is, [Studentas](#) &s)
- std::ostream & [operator<<](#) (std::ostream &os, const [Studentas](#) &s)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string [var_](#)
- string [pav_](#)

4.1.1 Constructor & Destructor Documentation

4.1.1.1 [Studentas\(\)](#) [1/5]

```
Studentas::Studentas ()
```

4.1.1.2 [Studentas\(\)](#) [2/5]

```
Studentas::Studentas (
    const string & vardas,
    const string & pavarde,
    const Vector< int > & paz,
    int egzaminas)
```

4.1.1.3 [Studentas\(\)](#) [3/5]

```
Studentas::Studentas (
    std::istream & is,
    int sk)
```

4.1.1.4 [Studentas\(\)](#) [4/5]

```
Studentas::Studentas (
    const Studentas & other)
```

4.1.1.5 [Studentas\(\)](#) [5/5]

```
Studentas::Studentas (
    Studentas && other) [noexcept]
```


4.1.1.6 ~Studentas()

```
Studentas::~~Studentas ()
```

4.1.2 Member Function Documentation

4.1.2.1 egzaminas()

```
int Studentas::egzaminas () const [inline]
```

4.1.2.2 galutinis_mediana()

```
double Studentas::galutinis_mediana () const [inline]
```

4.1.2.3 galutinis_vidurkis()

```
double Studentas::galutinis_vidurkis () const [inline]
```

4.1.2.4 ivedimas_generuojant_viska()

```
void Studentas::ivedimas_generuojant_viska ()
```

4.1.2.5 ivedimas_pazymiu_generavimu()

```
void Studentas::ivedimas_pazymiu_generavimu ()
```

4.1.2.6 ivedimas_ranka()

```
void Studentas::ivedimas_ranka ()
```

4.1.2.7 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other)
```

4.1.2.8 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [noexcept]
```

4.1.2.9 paskaiciuoti_gal()

```
void Studentas::paskaiciuoti_gal ()
```

4.1.2.10 paskaiciuoti_vid_ir_med()

```
void Studentas::paskaiciuoti_vid_ir_med ()
```

4.1.2.11 pavarde()

```
string Studentas::pavarde () const [inline], [virtual]
```

Implements [Zmogus](#).

4.1.2.12 spausdinti()

```
void Studentas::spausdinti (  
    std::ostream & os) const [override], [virtual]
```

Implements [Zmogus](#).

4.1.2.13 sugeneruoti_lyti()

```
void Studentas::sugeneruoti_lyti ()
```

4.1.2.14 sugeneruoti_pavarde()

```
void Studentas::sugeneruoti_pavarde ()
```

4.1.2.15 sugeneruoti_varda()

```
void Studentas::sugeneruoti_varda ()
```

4.1.2.16 vardas()

```
string Studentas::vardas () const [inline], [virtual]
```

Implements [Zmogus](#).

4.1.3 Friends And Related Symbol Documentation

4.1.3.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Studentas & s) [friend]
```

4.1.3.2 operator>>

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s) [friend]
```

The documentation for this class was generated from the following files:

- [vector/studentas.h](#)
- [vector/studentas.cpp](#)

4.2 Vector< T, Alloc > Class Template Reference

```
#include <Vector.hpp>
```

Public Types

- using [value_type](#) = T
- using [allocator_type](#) = Alloc
- using [size_type](#) = std::size_t
- using [difference_type](#) = std::ptrdiff_t
- using [reference](#) = T&
- using [const_reference](#) = T const&
- using [pointer](#) = typename std::allocator_traits<Alloc>::pointer
- using [const_pointer](#) = typename std::allocator_traits<Alloc>::const_pointer
- using [iterator](#) = T*
- using [const_iterator](#) = T const*
- using [reverse_iterator](#) = std::reverse_iterator<[iterator](#)>
- using [const_reverse_iterator](#) = std::reverse_iterator<[const_iterator](#)>

Public Member Functions

- [Vector](#) () noexcept(std::is_nothrow_default_constructible< Alloc >::value)
- [Vector](#) ([size_type](#) n, T const &value=T(), Alloc const &a=Alloc())
- template<typename InputIt, typename = typename std::enable_if<!std::is_integral<InputIt>::value>::type>
[Vector](#) (InputIt first, InputIt last, Alloc const &a=Alloc())
- [Vector](#) ([Vector](#) const &other)
- [Vector](#) ([Vector](#) &&other) noexcept
- [Vector](#) (std::initializer_list< T > il, Alloc const &a=Alloc())
- [~Vector](#) ()
- [Vector](#) & [operator=](#) ([Vector](#) const &rhs)
- [Vector](#) & [operator=](#) ([Vector](#) &&rhs) noexcept
- [iterator](#) [begin](#) () noexcept
- [const_iterator](#) [begin](#) () const noexcept
- [const_iterator](#) [cbegin](#) () const noexcept
- [iterator](#) [end](#) () noexcept
- [const_iterator](#) [end](#) () const noexcept
- [const_iterator](#) [cend](#) () const noexcept
- [reverse_iterator](#) [rbegin](#) () noexcept
- [const_reverse_iterator](#) [rbegin](#) () const noexcept

- [const_reverse_iterator crbegin](#) () const noexcept
- [reverse_iterator rend](#) () noexcept
- [const_reverse_iterator rend](#) () const noexcept
- [const_reverse_iterator crend](#) () const noexcept
- [bool empty](#) () const noexcept
- [size_type size](#) () const noexcept
- [size_type capacity](#) () const noexcept
- [void reserve](#) ([size_type](#) newCap)
- [void shrink_to_fit](#) ()
- [reference operator\[\]](#) ([size_type](#) i) noexcept
- [const_reference operator\[\]](#) ([size_type](#) i) const noexcept
- [reference at](#) ([size_type](#) i)
- [const_reference at](#) ([size_type](#) i) const
- [reference front](#) () noexcept
- [const_reference front](#) () const noexcept
- [reference back](#) () noexcept
- [const_reference back](#) () const noexcept
- [pointer data](#) () noexcept
- [const_pointer data](#) () const noexcept
- [void clear](#) () noexcept
- [void push_back](#) (T const &value)
- [void push_back](#) (T &&value)
- [template<typename... Args>](#)
[reference emplace_back](#) (Args &&... args)
- [void pop_back](#) ()
- [iterator insert](#) ([const_iterator](#) pos, T const &value)
- [iterator insert](#) ([const_iterator](#) pos, T &&value)
- [template<typename... Args>](#)
[iterator emplace](#) ([const_iterator](#) cpos, Args &&... args)
- [iterator erase](#) ([const_iterator](#) cpos)
- [iterator erase](#) ([const_iterator](#) cfirst, [const_iterator](#) clast)
- [void resize](#) ([size_type](#) n, T const &value=T())
- [void swap](#) ([Vector](#) &other) noexcept(std::allocator_traits< [Alloc](#) >::propagate_on_container_swap↵
::value||std::allocator_traits< [Alloc](#) >::is_always_equal::value)
- [template<typename InputIt>](#)
[void assign](#) (InputIt first, InputIt last)

4.2.1 Member Typedef Documentation

4.2.1.1 allocator_type

```
template<typename T, typename Alloc = std::allocator<T>>
using Vector< T, Alloc >::allocator_type = Alloc
```

4.2.1.2 const_iterator

```
template<typename T, typename Alloc = std::allocator<T>>
using Vector< T, Alloc >::const_iterator = T const*
```

4.2.1.3 const_pointer

```
template<typename T, typename Alloc = std::allocator<T>>
using Vector< T, Alloc >::const_pointer = typename std::allocator_traits<Alloc>::const_↵
pointer
```

4.2.1.4 const_reference

```
template<typename T, typename Alloc = std::allocator<T>>
using Vector< T, Alloc >::const_reference = T const&
```

4.2.1.5 const_reverse_iterator

```
template<typename T, typename Alloc = std::allocator<T>>
using Vector< T, Alloc >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

4.2.1.6 difference_type

```
template<typename T, typename Alloc = std::allocator<T>>
using Vector< T, Alloc >::difference_type = std::ptrdiff_t
```

4.2.1.7 iterator

```
template<typename T, typename Alloc = std::allocator<T>>
using Vector< T, Alloc >::iterator = T*
```

4.2.1.8 pointer

```
template<typename T, typename Alloc = std::allocator<T>>
using Vector< T, Alloc >::pointer = typename std::allocator_traits<Alloc>::pointer
```

4.2.1.9 reference

```
template<typename T, typename Alloc = std::allocator<T>>
using Vector< T, Alloc >::reference = T&
```

4.2.1.10 reverse_iterator

```
template<typename T, typename Alloc = std::allocator<T>>
using Vector< T, Alloc >::reverse_iterator = std::reverse_iterator<iterator>
```

4.2.1.11 size_type

```
template<typename T, typename Alloc = std::allocator<T>>
using Vector< T, Alloc >::size_type = std::size_t
```

4.2.1.12 value_type

```
template<typename T, typename Alloc = std::allocator<T>>
using Vector< T, Alloc >::value_type = T
```

4.2.2 Constructor & Destructor Documentation

4.2.2.1 Vector() [1/6]

```
template<typename T, typename Alloc = std::allocator<T>>
Vector< T, Alloc >::Vector () [inline], [noexcept]
```

4.2.2.2 Vector() [2/6]

```
template<typename T, typename Alloc = std::allocator<T>>
Vector< T, Alloc >::Vector (
    size_type n,
    T const & value = T(),
    Alloc const & a = Alloc()) [inline], [explicit]
```

4.2.2.3 Vector() [3/6]

```
template<typename T, typename Alloc = std::allocator<T>>
template<typename InputIt, typename = typename std::enable_if<!std::is_integral<InputIt>>←
::value>::type>
Vector< T, Alloc >::Vector (
    InputIt first,
    InputIt last,
    Alloc const & a = Alloc()) [inline]
```

4.2.2.4 Vector() [4/6]

```
template<typename T, typename Alloc = std::allocator<T>>
Vector< T, Alloc >::Vector (
    Vector< T, Alloc > const & other) [inline]
```

4.2.2.5 Vector() [5/6]

```
template<typename T, typename Alloc = std::allocator<T>>
Vector< T, Alloc >::Vector (
    Vector< T, Alloc > && other) [inline], [noexcept]
```

4.2.2.6 Vector() [6/6]

```
template<typename T, typename Alloc = std::allocator<T>>
Vector< T, Alloc >::Vector (
    std::initializer_list< T > il,
    Alloc const & a = Alloc()) [inline]
```

4.2.2.7 ~Vector()

```
template<typename T, typename Alloc = std::allocator<T>>
Vector< T, Alloc >::~~Vector () [inline]
```

4.2.3 Member Function Documentation

4.2.3.1 assign()

```
template<typename T, typename Alloc = std::allocator<T>>
template<typename InputIt>
void Vector< T, Alloc >::assign (
    InputIt first,
    InputIt last) [inline]
```

4.2.3.2 at() [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
reference Vector< T, Alloc >::at (
    size_type i) [inline]
```

4.2.3.3 at() [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
const_reference Vector< T, Alloc >::at (
    size_type i) const [inline]
```

4.2.3.4 back() [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
const_reference Vector< T, Alloc >::back () const [inline], [noexcept]
```

4.2.3.5 back() [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
reference Vector< T, Alloc >::back () [inline], [noexcept]
```

4.2.3.6 begin() [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
const_iterator Vector< T, Alloc >::begin () const [inline], [noexcept]
```

4.2.3.7 begin() [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
iterator Vector< T, Alloc >::begin () [inline], [noexcept]
```

4.2.3.8 capacity()

```
template<typename T, typename Alloc = std::allocator<T>>
size_type Vector< T, Alloc >::capacity () const [inline], [noexcept]
```

4.2.3.9 cbegin()

```
template<typename T, typename Alloc = std::allocator<T>>
const_iterator Vector< T, Alloc >::cbegin () const [inline], [noexcept]
```

4.2.3.10 cend()

```
template<typename T, typename Alloc = std::allocator<T>>
const_iterator Vector< T, Alloc >::cend () const [inline], [noexcept]
```

4.2.3.11 clear()

```
template<typename T, typename Alloc = std::allocator<T>>
void Vector< T, Alloc >::clear () [inline], [noexcept]
```

4.2.3.12 crbegin()

```
template<typename T, typename Alloc = std::allocator<T>>
const_reverse_iterator Vector< T, Alloc >::crbegin () const [inline], [noexcept]
```

4.2.3.13 crend()

```
template<typename T, typename Alloc = std::allocator<T>>
const_reverse_iterator Vector< T, Alloc >::crend () const [inline], [noexcept]
```

4.2.3.14 data() [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
const_pointer Vector< T, Alloc >::data () const [inline], [noexcept]
```

4.2.3.15 data() [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
pointer Vector< T, Alloc >::data () [inline], [noexcept]
```


4.2.3.16 `emplace()`

```
template<typename T, typename Alloc = std::allocator<T>>
template<typename... Args>
iterator Vector< T, Alloc >::emplace (
    const_iterator cpos,
    Args &&... args) [inline]
```

4.2.3.17 `emplace_back()`

```
template<typename T, typename Alloc = std::allocator<T>>
template<typename... Args>
reference Vector< T, Alloc >::emplace_back (
    Args &&... args) [inline]
```

4.2.3.18 `empty()`

```
template<typename T, typename Alloc = std::allocator<T>>
bool Vector< T, Alloc >::empty () const [inline], [noexcept]
```

4.2.3.19 `end()` [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
const_iterator Vector< T, Alloc >::end () const [inline], [noexcept]
```

4.2.3.20 `end()` [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
iterator Vector< T, Alloc >::end () [inline], [noexcept]
```

4.2.3.21 `erase()` [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
iterator Vector< T, Alloc >::erase (
    const_iterator cfirst,
    const_iterator clast) [inline]
```

4.2.3.22 `erase()` [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
iterator Vector< T, Alloc >::erase (
    const_iterator cpos) [inline]
```

4.2.3.23 `front()` [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
const_reference Vector< T, Alloc >::front () const [inline], [noexcept]
```

4.2.3.24 front() [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
reference Vector< T, Alloc >::front () [inline], [noexcept]
```

4.2.3.25 insert() [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
iterator Vector< T, Alloc >::insert (
    const_iterator pos,
    T && value) [inline]
```

4.2.3.26 insert() [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
iterator Vector< T, Alloc >::insert (
    const_iterator pos,
    T const & value) [inline]
```

4.2.3.27 operator=() [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
Vector & Vector< T, Alloc >::operator= (
    Vector< T, Alloc > && rhs) [inline], [noexcept]
```

4.2.3.28 operator=() [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
Vector & Vector< T, Alloc >::operator= (
    Vector< T, Alloc > const & rhs) [inline]
```

4.2.3.29 operator[]() [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
const_reference Vector< T, Alloc >::operator[] (
    size_type i) const [inline], [noexcept]
```

4.2.3.30 operator[]() [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
reference Vector< T, Alloc >::operator[] (
    size_type i) [inline], [noexcept]
```

4.2.3.31 pop_back()

```
template<typename T, typename Alloc = std::allocator<T>>
void Vector< T, Alloc >::pop_back () [inline]
```

4.2.3.32 push_back() [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
void Vector< T, Alloc >::push_back (
    T && value) [inline]
```

4.2.3.33 push_back() [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
void Vector< T, Alloc >::push_back (
    T const & value) [inline]
```

4.2.3.34 rbegin() [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
const_reverse_iterator Vector< T, Alloc >::rbegin () const [inline], [noexcept]
```

4.2.3.35 rbegin() [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
reverse_iterator Vector< T, Alloc >::rbegin () [inline], [noexcept]
```

4.2.3.36 rend() [1/2]

```
template<typename T, typename Alloc = std::allocator<T>>
const_reverse_iterator Vector< T, Alloc >::rend () const [inline], [noexcept]
```

4.2.3.37 rend() [2/2]

```
template<typename T, typename Alloc = std::allocator<T>>
reverse_iterator Vector< T, Alloc >::rend () [inline], [noexcept]
```

4.2.3.38 reserve()

```
template<typename T, typename Alloc = std::allocator<T>>
void Vector< T, Alloc >::reserve (
    size_type newCap) [inline]
```

4.2.3.39 resize()

```
template<typename T, typename Alloc = std::allocator<T>>
void Vector< T, Alloc >::resize (
    size_type n,
    T const & value = T()) [inline]
```

4.2.3.40 shrink_to_fit()

```
template<typename T, typename Alloc = std::allocator<T>>
void Vector< T, Alloc >::shrink_to_fit () [inline]
```

4.2.3.41 size()

```
template<typename T, typename Alloc = std::allocator<T>>
size_type Vector< T, Alloc >::size () const [inline], [noexcept]
```

4.2.3.42 swap()

```
template<typename T, typename Alloc = std::allocator<T>>
void Vector< T, Alloc >::swap (
    Vector< T, Alloc > & other) [inline], [noexcept]
```

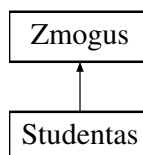
The documentation for this class was generated from the following file:

- vector/[Vector.hpp](#)

4.3 Zmogus Class Reference

```
#include <zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) ()=default
- [Zmogus](#) (const string &[vardas](#), const string &[pavarde](#))
- virtual string [vardas](#) () const =0
- virtual string [pavarde](#) () const =0
- virtual void [spausdinti](#) (std::ostream &os) const =0
- virtual [~Zmogus](#) ()

Protected Attributes

- string [var_](#)
- string [pav_](#)

4.3.1 Constructor & Destructor Documentation

4.3.1.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [default]
```

4.3.1.2 Zmogus() [2/2]

```
Zmogus::Zmogus (  
    const string & vardas,  
    const string & pavarde) [inline]
```

4.3.1.3 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

4.3.2 Member Function Documentation

4.3.2.1 pavarde()

```
virtual string Zmogus::pavarde () const [pure virtual]
```

Implemented in [Studentas](#).

4.3.2.2 spausdinti()

```
virtual void Zmogus::spausdinti (  
    std::ostream & os) const [pure virtual]
```

Implemented in [Studentas](#).

4.3.2.3 vardas()

```
virtual string Zmogus::vardas () const [pure virtual]
```

Implemented in [Studentas](#).

4.3.3 Member Data Documentation

4.3.3.1 pav_

```
string Zmogus::pav_ [protected]
```

4.3.3.2 var_

```
string Zmogus::var_ [protected]
```

The documentation for this class was generated from the following file:

- [vector/zmogus.h](#)

Chapter 5

File Documentation

5.1 vector/failu_funkcijos.cpp File Reference

```
#include "failu_funkcijos.h"
```

Functions

- int [rasti_nd_skaiciu_faile](#) (ifstream &fd)
- void [buferio_apdorojimas](#) (Vector< [Studentas](#) > &grupe, size_t buffer_size, Vector< char > &buffer, ifstream &fd, int nd_sk)
- void [failo_generavimas](#) (ifstream &fd)
- void [failo_pav_gavimas](#) (ifstream &fd)
- void [ivedimas_failu](#) (Vector< [Studentas](#) > &grupe)

5.1.1 Function Documentation

5.1.1.1 buferio_apdorojimas()

```
void buferio_apdorojimas (  
    Vector< Studentas > & grupe,  
    size_t buffer_size,  
    Vector< char > & buffer,  
    ifstream & fd,  
    int nd_sk)
```

5.1.1.2 failo_generavimas()

```
void failo_generavimas (  
    ifstream & fd)
```

5.1.1.3 failo_pav_gavimas()

```
void failo_pav_gavimas (  
    ifstream & fd)
```

5.1.1.4 ivedimas_failu()

```
void ivedimas_failu (  
    Vector< Studentas > & grupe)
```

5.1.1.5 rasti_nd_skaiciu_faile()

```
int rasti_nd_skaiciu_faile (  
    ifstream & fd)
```

5.2 vector/failu_funkcijos.h File Reference

```
#include "studentas.h"  
#include "spausdinimo_funkcijos.h"
```

Functions

- int `rasti_nd_skaiciu_faile` (ifstream &fd)
- void `buferio_apdorojimas` (Vector< Studentas > &grupe, size_t buffer_size, Vector< char > &buffer, ifstream &fd, int nd_sk)
- void `failo_generavimas` (ifstream &fd)
- void `failo_pav_gavimas` (ifstream &fd)
- void `ivedimas_failu` (Vector< Studentas > &grupe)

5.2.1 Function Documentation

5.2.1.1 buferio_apdorojimas()

```
void buferio_apdorojimas (  
    Vector< Studentas > & grupe,  
    size_t buffer_size,  
    Vector< char > & buffer,  
    ifstream & fd,  
    int nd_sk)
```

5.2.1.2 failo_generavimas()

```
void failo_generavimas (  
    ifstream & fd)
```

5.2.1.3 failo_pav_gavimas()

```
void failo_pav_gavimas (  
    ifstream & fd)
```


5.2.1.4 ivedimas_failu()

```
void ivedimas_failu (  
    Vector< Studentas > & grupe)
```

5.2.1.5 rasti_nd_skaiciu_faile()

```
int rasti_nd_skaiciu_faile (  
    ifstream & fd)
```

5.3 failu_funkcijos.h

[Go to the documentation of this file.](#)

```
00001 #ifndef APDOROJIMO_H  
00002 #define APDOROJIMO_H  
00003  
00004 #include "studentas.h"  
00005 #include "spausdinimo_funkcijos.h"  
00006 int rasti_nd_skaiciu_faile(ifstream& fd);  
00007 void buferio_apdorojimas(Vector <Studentas> &grupe, size_t buffer_size, Vector <char> &buffer,  
    ifstream &fd, int nd_sk);  
00008 void failo_generavimas(ifstream &fd);  
00009 void failo_pav_gavimas(ifstream &fd);  
00010 void ivedimas_failu(Vector <Studentas> &grupe);  
00011 #endif
```

5.4 vector/main.cpp File Reference

```
#include "mano_lib.h"  
#include "spausdinimo_funkcijos.h"  
#include "failu_funkcijos.h"  
#include "studentas.h"
```

Functions

- int [main](#) ()

5.4.1 Function Documentation

5.4.1.1 main()

```
int main ()
```

5.5 vector/mano_lib.h File Reference

```
#include <iostream>
#include <algorithm>
#include <cmath>
#include <iomanip>
#include <random>
#include <ctime>
#include <string>
#include <fstream>
#include <sstream>
#include <chrono>
#include <limits>
#include <exception>
#include <numeric>
#include "Vector.hpp"
```

5.6 mano_lib.h

[Go to the documentation of this file.](#)

```
00001 #ifndef MANO_BIBLIOTEKA
00002 #define MANO_BIBLIOTEKA
00003 #include <iostream>
00004 #include <algorithm>
00005 #include <cmath>
00006 #include <iomanip>
00007 #include <random>
00008 #include <ctime>
00009 #include <string>
00010 #include <fstream>
00011 #include <sstream>
00012 #include <chrono>
00013 #include <limits>
00014 #include <exception>
00015 #include <numeric>
00016 #include "Vector.hpp"
00017
00018 using std::stringstream;
00019 using std::string;
00020 using std::cout;
00021 using std::endl;
00022 using std::endl;
00023 using std::cin;
00024 using std::sort;
00025 using std::setw;
00026 using std::ifstream;
00027 using std::ofstream;
00028
00029 #endif
```

5.7 vector/spausdinimo_funkcijos.cpp File Reference

```
#include "spausdinimo_funkcijos.h"
```

Functions

- void [spausdinimas_atskiras](#) (Vector< [Studentas](#) > &grupe)
- void [spausdinimas_kartu](#) (Vector< [Studentas](#) > &grupe, bool spausdinimas)
- int [eigos_parinktis](#) ()

5.7.1 Function Documentation

5.7.1.1 eigos_parinktis()

```
int eigos_parinktis ()
```

5.7.1.2 spausdinimas_atskiras()

```
void spausdinimas_atskiras (  
    Vector< Studentas > & grupe)
```

5.7.1.3 spausdinimas_kartu()

```
void spausdinimas_kartu (  
    Vector< Studentas > & grupe,  
    bool spausdinimas)
```

5.8 vector/spausdinimo_funkcijos.h File Reference

```
#include "studentas.h"
```

Functions

- void `spausdinimas_kartu` (Vector< Studentas > &grupe, bool spausdinimas)
- void `spausdinimas_atskiras` (Vector< Studentas > &grupe)
- int `eigos_parinktis` ()

5.8.1 Function Documentation

5.8.1.1 eigos_parinktis()

```
int eigos_parinktis ()
```

5.8.1.2 spausdinimas_atskiras()

```
void spausdinimas_atskiras (  
    Vector< Studentas > & grupe)
```

5.8.1.3 spausdinimas_kartu()

```
void spausdinimas_kartu (  
    Vector< Studentas > & grupe,  
    bool spausdinimas)
```

5.9 spausdinimo_funkcijos.h

[Go to the documentation of this file.](#)

```
00001 #ifndef spausdinimo_H
00002 #define spausdinimo_H
00003
00004 #include "studentas.h"
00005
00006 void spausdinimas_kartu(Vector<Studentas> &grupe, bool spausdinimas);
00007 void spausdinimas_atskiras(Vector<Studentas> &grupe);
00008 int eigos_parinktis();
00009 #endif
```

5.10 vector/studentas.cpp File Reference

```
#include "studentas.h"
```

Functions

- std::istream & [operator>>](#) (std::istream &is, [Studentas](#) &s)
- std::ostream & [operator<<](#) (std::ostream &os, const [Studentas](#) &s)
- bool [lyginti_pagal_varda](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [lyginti_pagal_pavarde](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [lyginti_pagal_vidurki](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [lyginti_pagal_mediana](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- void [pasirink_rusiavimas](#) ([Vector](#)< [Studentas](#) > &grupe)
- void [rusiuoti_grupemis](#) ([Vector](#)< [Studentas](#) > &grupe, [Vector](#)< [Studentas](#) > &vargseliai)

5.10.1 Function Documentation

5.10.1.1 lyginti_pagal_mediana()

```
bool lyginti_pagal_mediana (
    const Studentas & a,
    const Studentas & b)
```

5.10.1.2 lyginti_pagal_pavarde()

```
bool lyginti_pagal_pavarde (
    const Studentas & a,
    const Studentas & b)
```

5.10.1.3 lyginti_pagal_varda()

```
bool lyginti_pagal_varda (
    const Studentas & a,
    const Studentas & b)
```

5.10.1.4 lyginti_pagal_vidurki()

```
bool lyginti_pagal_vidurki (  
    const Studentas & a,  
    const Studentas & b)
```

5.10.1.5 operator<<()

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Studentas & s)
```

5.10.1.6 operator>>()

```
std::istream & operator>> (  
    std::istream & is,  
    Studentas & s)
```

5.10.1.7 pasirink_rusiavimas()

```
void pasirink_rusiavimas (  
    Vector< Studentas > & grupe)
```

5.10.1.8 rusiuoti_grupemis()

```
void rusiuoti_grupemis (  
    Vector< Studentas > & grupe,  
    Vector< Studentas > & vargseliai)
```

5.11 vector/studentas.h File Reference

```
#include "mano_lib.h"  
#include "zmogus.h"
```

Classes

- class [Studentas](#)

Functions

- bool [lyginti_pagal_varda](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [lyginti_pagal_pavarda](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [lyginti_pagal_vidurki](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [lyginti_pagal_mediana](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- void [pasirink_rusiavimas](#) (Vector< [Studentas](#) > &grupe)
- void [rusiuoti_grupemis](#) (Vector< [Studentas](#) > &grupe, Vector< [Studentas](#) > &vargseliai)

5.11.1 Function Documentation

5.11.1.1 lyginti_pagal_mediana()

```
bool lyginti_pagal_mediana (  
    const Studentas & a,  
    const Studentas & b)
```

5.11.1.2 lyginti_pagal_pavarda()

```
bool lyginti_pagal_pavarda (  
    const Studentas & a,  
    const Studentas & b)
```

5.11.1.3 lyginti_pagal_varda()

```
bool lyginti_pagal_varda (  
    const Studentas & a,  
    const Studentas & b)
```

5.11.1.4 lyginti_pagal_vidurki()

```
bool lyginti_pagal_vidurki (  
    const Studentas & a,  
    const Studentas & b)
```

5.11.1.5 pasirink_rusiavimas()

```
void pasirink_rusiavimas (  
    Vector< Studentas > & grupe)
```

5.11.1.6 rusiuoti_grupemis()

```
void rusiuoti_grupemis (  
    Vector< Studentas > & grupe,  
    Vector< Studentas > & vargseliai)
```

5.12 studentas.h

[Go to the documentation of this file.](#)

```

00001 #ifndef student_H
00002 #define student_H
00003
00004 #include "mano_lib.h"
00005 #include "zmogus.h"
00006
00007 class Studentas : public Zmogus {
00008     private:
00009         int lytis_;
00010         Vector<int> paz_;
00011         int egz_;
00012         double vidurkis_;
00013         double mediana_;
00014         double galutinis_pagal_vid_;
00015         double galutinis_pagal_med_;
00016
00017     public:
00018         Studentas();
00019         Studentas(const string& vardas, const string& pavarde, const Vector<int>& paz, int egzaminas);
00020         Studentas(std::istream& is, int sk);
00021
00022         Studentas(const Studentas& other);
00023         Studentas(Studentas&& other) noexcept;
00024         Studentas& operator=(const Studentas& other);
00025         Studentas& operator=(Studentas&& other) noexcept;
00026         ~Studentas();
00027
00028         friend std::istream& operator>(std::istream& is, Studentas& s);
00029         friend std::ostream& operator<(std::ostream& os, const Studentas& s);
00030
00031         void ivedimas_ranka();
00032         void ivedimas_pazymiu_generavimu();
00033         void ivedimas_generuojant_viska();
00034         void paskaiciuoti_vid_ir_med();
00035         void paskaiciuoti_gal();
00036         void sugeneruoti_lyti();
00037         void sugeneruoti_varda();
00038         void sugeneruoti_pavarde();
00039
00040         string vardas() const { return var_; }
00041         string pavarde() const { return pav_; }
00042         int egzaminas() const { return egz_; }
00043         double galutinis_vidurkis() const { return galutinis_pagal_vid_; }
00044         double galutinis_mediana() const { return galutinis_pagal_med_; }
00045         void spausdinti(std::ostream& os) const override;
00046
00047 };
00048
00049 bool lyginti_pagal_varda(const Studentas &a, const Studentas &b);
00050 bool lyginti_pagal_pavarda(const Studentas &a, const Studentas &b);
00051 bool lyginti_pagal_vidurki(const Studentas &a, const Studentas &b);
00052 bool lyginti_pagal_mediana(const Studentas &a, const Studentas &b);
00053 void pasirink_rusiavimas(Vector<Studentas> &grupe);
00054 void rusiuoti_grupemis(Vector<Studentas> &grupe, Vector<Studentas> &vargseliai);
00055
00056 #endif

```

5.13 vector/Vector.hpp File Reference

```

#include <memory>
#include <initializer_list>
#include <iterator>
#include <algorithm>
#include <stdexcept>
#include <utility>
#include <ostream>

```

Classes

- class [Vector< T, Alloc >](#)

Functions

- `template<typename T, typename Alloc>`
`void swap (Vector< T, Alloc > &a, Vector< T, Alloc > &b) noexcept(noexcept(a.swap(b)))`
- `template<typename T, typename Alloc>`
`bool operator== (Vector< T, Alloc > const &a, Vector< T, Alloc > const &b)`
- `template<typename T, typename Alloc>`
`bool operator!= (Vector< T, Alloc > const &a, Vector< T, Alloc > const &b)`
- `template<typename T, typename Alloc>`
`std::ostream & operator<< (std::ostream &os, const Vector< T, Alloc > &v)`

5.13.1 Function Documentation

5.13.1.1 operator"!=()

```
template<typename T, typename Alloc>
bool operator!= (
    Vector< T, Alloc > const & a,
    Vector< T, Alloc > const & b)
```

5.13.1.2 operator<<()

```
template<typename T, typename Alloc>
std::ostream & operator<< (
    std::ostream & os,
    const Vector< T, Alloc > & v)
```

5.13.1.3 operator==()

```
template<typename T, typename Alloc>
bool operator== (
    Vector< T, Alloc > const & a,
    Vector< T, Alloc > const & b)
```

5.13.1.4 swap()

```
template<typename T, typename Alloc>
void swap (
    Vector< T, Alloc > & a,
    Vector< T, Alloc > & b) [noexcept]
```


5.14 Vector.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef VECTOR_HPP
00002 #define VECTOR_HPP
00003
00004 #include <memory>
00005 #include <initializer_list>
00006 #include <iterator>
00007 #include <algorithm>
00008 #include <stdexcept>
00009 #include <utility>
00010 #include <ostream>
00011
00012 template<typename T, typename Alloc = std::allocator<T>
00013 class Vector {
00014 public:
00015     // member types
00016     using value_type = T;
00017     using allocator_type = Alloc;
00018     using size_type = std::size_t;
00019     using difference_type = std::ptrdiff_t;
00020     using reference = T&;
00021     using const_reference = T const&;
00022     using pointer = typename std::allocator_traits<Alloc>::pointer;
00023     using const_pointer = typename std::allocator_traits<Alloc>::const_pointer;
00024     using iterator = T*;
00025     using const_iterator = T const*;
00026     using reverse_iterator = std::reverse_iterator<iterator>;
00027     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00028
00029     // constructors
00030     Vector() noexcept (std::is_nothrow_default_constructible<Alloc>::value) //default
00031         : alloc_(), data_(nullptr), size_(0), cap_(0) {}
00032
00033     explicit Vector(size_type n, T const& value = T(), Alloc const& a = Alloc()) //size+value
00034         : alloc_(a), size_(n), cap_(n)
00035     {
00036         data_ = alloc_.allocate(cap_);
00037         for (size_type i = 0; i < n; ++i)
00038             std::allocator_traits<Alloc>::construct(alloc_, data_ + i, value);
00039     }
00040
00041     template<typename InputIt, //range
00042             typename = typename std::enable_if<!std::is_integral<InputIt>::value>::type>
00043     Vector(InputIt first, InputIt last, Alloc const& a = Alloc())
00044         : alloc_(a)
00045     {
00046         size_type n = std::distance(first, last);
00047         data_ = alloc_.allocate(n);
00048         size_ = cap_ = n;
00049         std::uninitialized_copy(first, last, data_);
00050     }
00051
00052     Vector(Vector const& other) //copy
00053         : alloc_(std::allocator_traits<Alloc>::select_on_container_copy_construction(other.alloc_)),
00054           size_(other.size_), cap_(other.size_)
00055     {
00056         data_ = alloc_.allocate(cap_);
00057         std::uninitialized_copy(other.data_, other.data_ + size_, data_);
00058     }
00059
00060     Vector(Vector&& other) noexcept //move
00061         : alloc_(std::move(other.alloc_)),
00062           data_(other.data_), size_(other.size_), cap_(other.cap_)
00063     {
00064         other.data_ = nullptr;
00065         other.size_ = other.cap_ = 0;
00066     }
00067
00068     Vector(std::initializer_list<T> il, Alloc const& a = Alloc()) //init list
00069         : Vector(il.begin(), il.end(), a) {}
00070
00071     ~Vector() { //destr
00072         clear();
00073         if (data_) alloc_.deallocate(data_, cap_);
00074     }
00075
00076     //copy/move assign
00077     Vector& operator=(Vector const& rhs) {
00078         if (this != &rhs) {
00079             Vector tmp(rhs);
00080             swap(tmp);
00081         }
00082         return *this;

```

```

00083     }
00084
00085     Vector& operator=(Vector&& rhs) noexcept {
00086         if (this != &rhs) {
00087             clear();
00088             data_ = rhs.data_;
00089             size_ = rhs.size_;
00090             cap_ = rhs.cap_;
00091             rhs.data_ = nullptr;
00092             rhs.size_ = rhs.cap_ = 0;
00093         }
00094         return *this;
00095     }
00096
00097     // iterators
00098     iterator begin() noexcept { return data_; } //begin
00099     const_iterator begin() const noexcept { return data_; }
00100     const_iterator cbegin() const noexcept { return data_; }
00101
00102     iterator end() noexcept { return data_ + size_; } //end
00103     const_iterator end() const noexcept { return data_ + size_; }
00104     const_iterator cend() const noexcept { return data_ + size_; }
00105
00106     reverse_iterator rbegin() noexcept { return reverse_iterator(end()); }
00107     const_reverse_iterator rbegin() const noexcept { return const_reverse_iterator(end()); }
00108     const_reverse_iterator crbegin() const noexcept { return const_reverse_iterator(end()); }
00109
00110     reverse_iterator rend() noexcept { return reverse_iterator(begin()); } //reverse
00111     const_reverse_iterator rend() const noexcept { return const_reverse_iterator(begin()); }
00112     const_reverse_iterator crend() const noexcept { return const_reverse_iterator(begin()); }
00113
00114     //capacity
00115     bool empty() const noexcept { return size_ == 0; }
00116     size_type size() const noexcept { return size_; }
00117     size_type capacity() const noexcept { return cap_; }
00118
00119     void reserve(size_type newCap) {
00120         if (newCap <= cap_) return;
00121         pointer newData = alloc_.allocate(newCap);
00122         for (size_type i = 0; i < size_; ++i) {
00123             std::allocator_traits<Alloc>::construct(alloc_, newData + i,
00124                 std::move_if_noexcept(data_[i]));
00125             std::allocator_traits<Alloc>::destroy(alloc_, data_ + i);
00126         }
00127         if (data_) alloc_.deallocate(data_, cap_);
00128         data_ = newData;
00129         cap_ = newCap;
00130     }
00131
00132     void shrink_to_fit() {
00133         if (size_ < cap_) {
00134             pointer newData = alloc_.allocate(size_);
00135             for (size_type i = 0; i < size_; ++i) {
00136                 std::allocator_traits<Alloc>::construct(alloc_, newData + i,
00137                     std::move_if_noexcept(data_[i]));
00138                 std::allocator_traits<Alloc>::destroy(alloc_, data_ + i);
00139             }
00140             alloc_.deallocate(data_, cap_);
00141             data_ = newData;
00142             cap_ = size_;
00143         }
00144
00145         //element access
00146         reference operator[](size_type i) noexcept { return data_[i]; } //unchecked
00147         const_reference operator[](size_type i) const noexcept { return data_[i]; }
00148
00149         reference at(size_type i) { //checked
00150             if (i >= size_) throw std::out_of_range("Vector::at");
00151             return data_[i];
00152         }
00153         const_reference at(size_type i) const {
00154             if (i >= size_) throw std::out_of_range("Vector::at");
00155             return data_[i];
00156         }
00157
00158         reference front() noexcept { return data_[0]; } //front/back
00159         const_reference front() const noexcept { return data_[0]; }
00160         reference back() noexcept { return data_[size_-1]; }
00161         const_reference back() const noexcept { return data_[size_-1]; }
00162
00163         pointer data() noexcept { return data_; } //raw ptr
00164         const_pointer data() const noexcept { return data_; }
00165
00166         //modifiers
00167         void clear() noexcept {
00168             for (size_type i = 0; i < size_; ++i)

```

```

00169         std::allocator_traits<Alloc>::destroy(alloc_, data_ + i);
00170         size_ = 0;
00171     }
00172
00173     void push_back(T const& value) {
00174         if (size_ == cap_) reserve(cap_ ? cap_*10 : 1);
00175         std::allocator_traits<Alloc>::construct(alloc_, data_ + size_++, value);
00176     }
00177     void push_back(T& value) {
00178         if (size_ == cap_) reserve(cap_ ? cap_*10 : 1);
00179         std::allocator_traits<Alloc>::construct(alloc_, data_ + size_++, std::move(value));
00180     }
00181
00182     template<typename... Args>
00183     reference emplace_back(Args&&... args) {
00184         if (size_ == cap_) reserve(cap_ ? cap_*10 : 1);
00185         std::allocator_traits<Alloc>::construct(alloc_, data_ + size_, std::forward<Args>(args)...);
00186         return data_[size_++];
00187     }
00188
00189     void pop_back() {
00190         if (size_) {
00191             std::allocator_traits<Alloc>::destroy(alloc_, data_ + --size_);
00192         }
00193     }
00194
00195     iterator insert(const_iterator pos, T const& value) {
00196         return emplace(pos, value);
00197     }
00198     iterator insert(const_iterator pos, T&& value) {
00199         return emplace(pos, std::move(value));
00200     }
00201
00202     template<typename... Args>
00203     iterator emplace(const_iterator cpos, Args&&... args) {
00204         size_type idx = cpos - data_;
00205         if (size_ == cap_) reserve(cap_ ? cap_*2 : 1);
00206         iterator pos = data_ + idx;
00207         // move elements one step right
00208         for (iterator it = data_ + size_; it != pos; --it) {
00209             std::allocator_traits<Alloc>::construct(
00210                 alloc_, it, std::move(*(it-1)));
00211             std::allocator_traits<Alloc>::destroy(alloc_, it-1);
00212         }
00213         std::allocator_traits<Alloc>::construct(alloc_, pos, std::forward<Args>(args)...);
00214         ++size_;
00215         return pos;
00216     }
00217
00218     iterator erase(const_iterator cpos) {
00219         return erase(cpos, cpos+1);
00220     }
00221     iterator erase(const_iterator cfirst, const_iterator clast) {
00222         size_type idx1 = cfirst - data_;
00223         size_type idx2 = clast - data_;
00224         for (size_type i = idx2; i < size_; ++i) {
00225             data_[idx1 + i - idx2] = std::move(data_[i]);
00226         }
00227         for (size_type i = size_ - (idx2 - idx1); i < size_; ++i) {
00228             std::allocator_traits<Alloc>::destroy(alloc_, data_ + i);
00229         }
00230         size_ -= (idx2 - idx1);
00231         return data_ + idx1;
00232     }
00233
00234     void resize(size_type n, T const& value = T()) {
00235         if (n < size_) {
00236             while (size_ > n) pop_back();
00237         } else {
00238             reserve(n);
00239             while (size_ < n) push_back(value);
00240         }
00241     }
00242
00243     void swap(Vector& other) noexcept {
00244         std::allocator_traits<Alloc>::propagate_on_container_swap::value ||
00245         std::allocator_traits<Alloc>::is_always_equal::value)
00246     {
00247         using std::swap;
00248         swap(alloc_, other.alloc_);
00249         swap(data_, other.data_);
00250         swap(size_, other.size_);
00251         swap(cap_, other.cap_);
00252     }
00253
00254     template<typename InputIt>
00255     void assign(InputIt first, InputIt last) {

```

```

00256         clear();
00257         reserve(std::distance(first, last));
00258         for (; first != last; ++first)
00259             push_back(*first);
00260     }
00261
00262 private:
00263     Alloc alloc_;
00264     pointer data_;
00265     size_type size_;
00266     size_type cap_;
00267 };
00268
00269 // non-member functions
00270 template<typename T, typename Alloc>
00271 void swap(Vector<T,Alloc>& a, Vector<T,Alloc>& b) noexcept(noexcept(a.swap(b))) {
00272     a.swap(b);
00273 }
00274
00275 template<typename T, typename Alloc>
00276 bool operator==(Vector<T,Alloc> const& a, Vector<T,Alloc> const& b){
00277     return a.size()==b.size() && std::equal(a.begin(), a.end(), b.begin());
00278 }
00279
00280 template<typename T, typename Alloc>
00281 bool operator!=(Vector<T,Alloc> const& a, Vector<T,Alloc> const& b){ return !(a==b); }
00282
00283 template<typename T, typename Alloc>
00284 std::ostream& operator<<(std::ostream& os, const Vector<T, Alloc>& v) {
00285     os << "[";
00286     for (std::size_t i = 0; i < v.size(); ++i) {
00287         os << v[i];
00288         if (i + 1 != v.size()) os << ", ";
00289     }
00290     os << "]";
00291     return os;
00292 }
00293
00294 #endif

```

5.15 vector/zmogus.h File Reference

```
#include <string>
```

Classes

- class [Zmogus](#)

5.16 zmogus.h

[Go to the documentation of this file.](#)

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005 using std::string;
00006
00007 class Zmogus {
00008 protected:
00009     string var_, pav_;
00010
00011 public:
00012     Zmogus() = default;
00013     Zmogus(const string& vardas, const string& pavarde) : var_(vardas), pav_(pavarde) {}
00014
00015     virtual string vardas() const = 0;
00016     virtual string pavarde() const = 0;
00017
00018     virtual void spausdinti(std::ostream& os) const = 0;
00019     virtual ~Zmogus() {}
00020 };
00021
00022 #endif

```

Index

- ~Studentas
 - Studentas, [8](#)
- ~Vector
 - Vector< T, Alloc >, [14](#)
- ~Zmogus
 - Zmogus, [21](#)
- allocator_type
 - Vector< T, Alloc >, [12](#)
- assign
 - Vector< T, Alloc >, [15](#)
- at
 - Vector< T, Alloc >, [15](#)
- back
 - Vector< T, Alloc >, [15](#)
- begin
 - Vector< T, Alloc >, [15](#)
- buferio_apdorojimas
 - failu_funkcijos.cpp, [23](#)
 - failu_funkcijos.h, [24](#)
- capacity
 - Vector< T, Alloc >, [15](#)
- cbegin
 - Vector< T, Alloc >, [16](#)
- cend
 - Vector< T, Alloc >, [16](#)
- clear
 - Vector< T, Alloc >, [16](#)
- const_iterator
 - Vector< T, Alloc >, [12](#)
- const_pointer
 - Vector< T, Alloc >, [12](#)
- const_reference
 - Vector< T, Alloc >, [13](#)
- const_reverse_iterator
 - Vector< T, Alloc >, [13](#)
- crbegin
 - Vector< T, Alloc >, [16](#)
- crend
 - Vector< T, Alloc >, [16](#)
- data
 - Vector< T, Alloc >, [16](#)
- difference_type
 - Vector< T, Alloc >, [13](#)
- egzaminas
 - Studentas, [9](#)
- eigos_parinktis
 - spausdinimo_funkcijos.cpp, [27](#)
 - spausdinimo_funkcijos.h, [27](#)
- emplace
 - Vector< T, Alloc >, [16](#)
- emplace_back
 - Vector< T, Alloc >, [17](#)
- empty
 - Vector< T, Alloc >, [17](#)
- end
 - Vector< T, Alloc >, [17](#)
- erase
 - Vector< T, Alloc >, [17](#)
- failo_generavimas
 - failu_funkcijos.cpp, [23](#)
 - failu_funkcijos.h, [24](#)
- failo_pav_gavimas
 - failu_funkcijos.cpp, [23](#)
 - failu_funkcijos.h, [24](#)
- failu_funkcijos.cpp
 - buferio_apdorojimas, [23](#)
 - failo_generavimas, [23](#)
 - failo_pav_gavimas, [23](#)
 - ivedimas_failu, [23](#)
 - rasti_nd_skaiciu_faile, [24](#)
- failu_funkcijos.h
 - buferio_apdorojimas, [24](#)
 - failo_generavimas, [24](#)
 - failo_pav_gavimas, [24](#)
 - ivedimas_failu, [24](#)
 - rasti_nd_skaiciu_faile, [25](#)
- front
 - Vector< T, Alloc >, [17](#)
- galutinis_mediana
 - Studentas, [9](#)
- galutinis_vidurkis
 - Studentas, [9](#)
- insert
 - Vector< T, Alloc >, [18](#)
- iterator
 - Vector< T, Alloc >, [13](#)
- ivedimas_failu
 - failu_funkcijos.cpp, [23](#)
 - failu_funkcijos.h, [24](#)
- ivedimas_generuojant_viska
 - Studentas, [9](#)
- ivedimas_pazymiu_generavimu
 - Studentas, [9](#)

- ivedimas_ranka
 - Studentas, 9
- lyginti_pagal_mediana
 - studentas.cpp, 28
 - studentas.h, 30
- lyginti_pagal_pavarda
 - studentas.h, 30
- lyginti_pagal_pavarde
 - studentas.cpp, 28
- lyginti_pagal_varda
 - studentas.cpp, 28
 - studentas.h, 30
- lyginti_pagal_vidurki
 - studentas.cpp, 28
 - studentas.h, 30
- main
 - main.cpp, 25
- main.cpp
 - main, 25
- operator!=
 - Vector.hpp, 32
- operator<<
 - Studentas, 10
 - studentas.cpp, 29
 - Vector.hpp, 32
- operator>>
 - Studentas, 10
 - studentas.cpp, 29
- operator=
 - Studentas, 9
 - Vector< T, Alloc >, 18
- operator==
 - Vector.hpp, 32
- operator[]
 - Vector< T, Alloc >, 18
- pasirink_rusiavimas
 - studentas.cpp, 29
 - studentas.h, 30
- paskaiciuoti_gal
 - Studentas, 9
- paskaiciuoti_vid_ir_med
 - Studentas, 10
- pav_
 - Zmogus, 21
- pavarde
 - Studentas, 10
 - Zmogus, 21
- pointer
 - Vector< T, Alloc >, 13
- pop_back
 - Vector< T, Alloc >, 18
- push_back
 - Vector< T, Alloc >, 18, 19
- rasti_nd_skaiciu_faile
 - failu_funkcijos.cpp, 24
 - failu_funkcijos.h, 25
- rbegin
 - Vector< T, Alloc >, 19
- reference
 - Vector< T, Alloc >, 13
- rend
 - Vector< T, Alloc >, 19
- reserve
 - Vector< T, Alloc >, 19
- resize
 - Vector< T, Alloc >, 19
- reverse_iterator
 - Vector< T, Alloc >, 13
- rusiuoti_grupemis
 - studentas.cpp, 29
 - studentas.h, 30
- shrink_to_fit
 - Vector< T, Alloc >, 19
- size
 - Vector< T, Alloc >, 20
- size_type
 - Vector< T, Alloc >, 13
- spausdinimas_atskiras
 - spausdinimo_funkcijos.cpp, 27
 - spausdinimo_funkcijos.h, 27
- spausdinimas_kartu
 - spausdinimo_funkcijos.cpp, 27
 - spausdinimo_funkcijos.h, 27
- spausdinimo_funkcijos.cpp
 - eigos_parinktis, 27
 - spausdinimas_atskiras, 27
 - spausdinimas_kartu, 27
- spausdinimo_funkcijos.h
 - eigos_parinktis, 27
 - spausdinimas_atskiras, 27
 - spausdinimas_kartu, 27
- spausdinti
 - Studentas, 10
 - Zmogus, 21
- Studentas, 7
 - ~Studentas, 8
 - egzaminas, 9
 - galutinis_mediana, 9
 - galutinis_vidurkis, 9
 - ivedimas_generuojant_viska, 9
 - ivedimas_pazymiu_generavimu, 9
 - ivedimas_ranka, 9
 - operator<<, 10
 - operator>>, 10
 - operator=, 9
 - paskaiciuoti_gal, 9
 - paskaiciuoti_vid_ir_med, 10
 - pavarde, 10
 - spausdinti, 10
 - Studentas, 8
 - sugeneruoti_lyti, 10
 - sugeneruoti_pavarde, 10

- sugeneruoti_varda, 10
- vardas, 10
- studentas.cpp
 - lyginti_pagal_mediana, 28
 - lyginti_pagal_pavarde, 28
 - lyginti_pagal_varda, 28
 - lyginti_pagal_vidurki, 28
 - operator<<, 29
 - operator>>, 29
 - pasirink_rusiavimas, 29
 - rusiuoti_grupemis, 29
- studentas.h
 - lyginti_pagal_mediana, 30
 - lyginti_pagal_pavarda, 30
 - lyginti_pagal_varda, 30
 - lyginti_pagal_vidurki, 30
 - pasirink_rusiavimas, 30
 - rusiuoti_grupemis, 30
- sugeneruoti_lyti
 - Studentas, 10
- sugeneruoti_pavarde
 - Studentas, 10
- sugeneruoti_varda
 - Studentas, 10
- swap
 - Vector< T, Alloc >, 20
 - Vector.hpp, 32
- value_type
 - Vector< T, Alloc >, 13
- var_
 - Zmogus, 21
- vardas
 - Studentas, 10
 - Zmogus, 21
- Vector
 - Vector< T, Alloc >, 14
- Vector< T, Alloc >, 11
 - ~Vector, 14
 - allocator_type, 12
 - assign, 15
 - at, 15
 - back, 15
 - begin, 15
 - capacity, 15
 - cbegin, 16
 - cend, 16
 - clear, 16
 - const_iterator, 12
 - const_pointer, 12
 - const_reference, 13
 - const_reverse_iterator, 13
 - crbegin, 16
 - crend, 16
 - data, 16
 - difference_type, 13
 - emplace, 16
 - emplace_back, 17
 - empty, 17
 - end, 17
 - erase, 17
 - front, 17
 - insert, 18
 - iterator, 13
 - operator=, 18
 - operator[], 18
 - pointer, 13
 - pop_back, 18
 - push_back, 18, 19
 - rbegin, 19
 - reference, 13
 - rend, 19
 - reserve, 19
 - resize, 19
 - reverse_iterator, 13
 - shrink_to_fit, 19
 - size, 20
 - size_type, 13
 - swap, 20
 - value_type, 13
 - Vector, 14
- Vector.hpp
 - operator!=, 32
 - operator<<, 32
 - operator==, 32
 - swap, 32
- vector/failu_funkcijos.cpp, 23
- vector/failu_funkcijos.h, 24, 25
- vector/main.cpp, 25
- vector/mano_lib.h, 26
- vector/spausdinimo_funkcijos.cpp, 26
- vector/spausdinimo_funkcijos.h, 27, 28
- vector/studentas.cpp, 28
- vector/studentas.h, 29, 31
- vector/Vector.hpp, 31, 33
- vector/zmogus.h, 36
- Zmogus, 20
 - ~Zmogus, 21
 - pav_, 21
 - pavarde, 21
 - spausdinti, 21
 - var_, 21
 - vardas, 21
 - Zmogus, 21